

PakFasal — Complete Technical Documentation

Version: 1.0.0+1 (from pubspec.yaml)
Platform: Flutter (Dart SDK ^3.8.1)
Document date: 2026-05-09

Important: What This Repository Actually Contains

This documentation is based on **reading the full lib/ tree** (77 Dart files). Some items you asked about exist only as **design ideas or thesis text**, not as implemented features in code:

Topic	In this codebase
MySQL	Not used. There is no SQL database layer.
Dedicated Admin Panel (app screen)	Not present. Remote tuning of DSS rules is done via Firebase Console → Firestore document <code>sensor_config/rules</code> .
Push notifications (FCM)	Not implemented (<code>firebase_messaging</code> is not in <code>pubspec.yaml</code>). The home screen shows a static demo notification list in a bottom sheet.
Marketplace product upload / backend	Not implemented. Products come from an in-memory mock list ; <code>MarketplaceProvider.isOffline</code> is hard-coded <code>true</code> .
AI Q&A backend	Not implemented. <code>AiQueryScreen</code> shows a placeholder bot reply after a fixed delay; no LLM or REST API.
Crop calendar “recommendations”	Rule-based static catalog (month windows + localization keys), not ML and not from an API.

Everything below reflects **actual code behaviour**.

1. Project Purpose (Beginner Summary)

PakFasal is a smart-farming companion app aimed at Pakistani farmers. It combines:

- **Account access** (Firebase email/password, optional guest mode).
- **Weather** from external HTTP APIs with local caching.
- **Learning** videos via YouTube Data API when a key is configured, else demo data.
- **Crop calendar** timelines from a **static Punjab-focused catalog**.
- **Marketplace** browsing of **demo products** (filter/search/favourites only).
- **Sensors + DSS:** manual entry of soil moisture and pH; rules combine **Firestore thresholds**, **weather rain chance**, and **client-side logic** to produce irrigation/soil/risk/monitoring text; history syncs to **Firestore**.

- **AI chat UI** as a **prototype** (voice input works; answers are stubbed).

2. High-Level Architecture

Pattern: Feature-first **layered architecture** (not strict MVC). Each feature uses:

- `domain/` — plain Dart entities.
- `data/` — repositories, services, catalog/mock JSON-like data.
- `presentation/` — screens, widgets, `ChangeNotifier` providers.

Global state: `provider` package — `MultiProvider` in `main.dart`.

3. Folder Structure

```
lib/ main.dart # Firebase + Hive init, providers, runApp firebase_options.dart #
FlutterFire-generated Firebase config src/ app.dart # MaterialApp, routes, theme/locale core/
config/app_config.dart # Secrets & URLs (dart-define + JSON asset) error/error_logger.dart #
Crashlytics facade localization/ # EN/UR strings + controller routing/app_routes.dart # Named routes
+ AuthGate wrapping theme/ # Colors, light/dark themes, ThemeController widgets/ # AuthGate,
scaffolds, shared UI features/ ai_query/ # Chat UI (stub responses) auth/ # Login, signup, forgot
password, session crop_calendar/ # Static catalog + timeline UI home/ # Dashboard + bottom nav shell
learning/ # YouTube + disease catalog marketplace/ # Mock products onboarding/ profile/ sensor/ #
Firestore + DSS rules + charts splash/ weather/ # Full weather stack
```

4. Runtime Initialization (`main.dart`)

Order of operations:

1. `WidgetsFlutterBinding.ensureInitialized()`
2. `AppConfig.init()` — loads `config/app_config.json` asset (optional overrides).
3. `Firebase.initializeApp` with `DefaultFirebaseOptions`.
4. `ErrorLogger.instance.init()` — Crashlytics hooks (`FlutterError`, `PlatformDispatcher`).
5. `Hive.initFlutter()` and open boxes:
6. `weather_cache`
7. `learning_cache`

8. `app_preferences` (locale, theme, onboarding flag)
 9. `AuthSessionController` — listens to `authStateChanges`; syncs user id to Crashlytics.
 10. `MultiProvider` registers:
 11. `AuthSessionController`
 12. `LocalizationController` (proxy from auth user id)
 13. `ThemeController` (proxy from auth user id)
 14. `WeatherProvider` with `startAutoRefresh()` (periodic refresh)
 15. `CropCalendarProvider`
 16. `runZonedGuarded` sends uncaught async errors to `ErrorLogger`.
-

5. Routing & Authentication Gate

File: `lib/src/core/routing/app_routes.dart`

Routes: `/`, `/login`, `/signup`, `/forgot-password`, `/onboarding`, `/home`, `/learning`, `/weather`, `/weather/city-search`, `/ai-query`, `/sensor`, `/profile`, `/marketplace`, `/crop-calendar`.

AuthGate (`lib/src/core/widgets/auth_gate.dart`):

- Watches `AuthSessionController`.
- If `isSignedIn` → shows protected child (e.g. `HomeDashboardScreen`).
- Else → embeds `LoginScreen` **inline** (no redirect route).

Who counts as signed in: Firebase user **or** guest (`continueAsGuest()` sets `_isGuestUser`).

6. Module Reference

6.1 Authentication

Purpose: Email/password sign-in and registration via **Firebase Authentication**; password reset email; optional **Continue as guest**; profile display name updates.

Frontend flow:

1. **Splash** (`splash_screen.dart`): After ~5s, reads `Hive app_preferences` → `onboarding_seen`. If `false` → onboarding. Else if `AuthSessionController.isSignedIn` → `home`, else login.
2. **Login / Signup / Forgot password** screens use forms and call controller methods.
3. After success, navigation typically `pushNamedAndRemoveUntil` → `/home`.

Backend flow: All auth operations go through **Firebase Auth SDK** (`FirebaseAuth.instance`). No custom REST “auth API” in this repo.

Database: Firebase Auth stores users in **Firestore's identity backend** (not Firestore collections for credentials). The app does **not** mirror user profiles to Firestore in code.

State: `AuthSessionController` (`ChangeNotifier`): `_auth.authStateChanges()`, loading/error mapping via `_mapAuthError`, guest flag.

Validation: Form validators on screens (email format, password length/strength patterns where implemented — see `auth_form_provider.dart` if used).

Error handling: Exceptions mapped to localization keys (`invalidEmail`, `wrongPassword`, etc.).

6.2 Weather Module

Purpose: Show current conditions, hourly slider, daily forecast, alerts (when API provides them), farmer advisory hints, city search, saved cities.

Workflow (step by step):

1. `WeatherProvider.ensureLoaded()` OR `refreshAll()` runs.
2. `WeatherRepository.resolveActiveLocation()`:
3. If user previously picked a city (`SharedPreferences weather.selected_location`) and it is not “current GPS”, use it.
4. Else try GPS via `LocationService` → reverse geocode label via API.
5. Else use last cached snapshot location.
6. Else default **Lahore, Pakistan**.
7. `WeatherRepository.fetchSnapshot()` checks **Hive** cache key `snapshot_{lat_lon}`:
8. If fresh (< **10 minutes** TTL for snapshot — see repository `_isFresh`) and not forcing refresh → return cached.
9. Else `weatherApiService.fetchSnapshot()` calls external HTTP:
10. If `OPENWEATHER_USE_ONECALL_V3=true` **and** API key → OpenWeather **One Call 3.0**.

11. Else if OpenWeather API key → **2.5** current + 5-day/3-hour + air pollution.
12. Else → **Open-Meteo** (no key) + Open-Meteo air quality helper.
13. On success, snapshot JSON serialized to Hive; prefs updated.
14. On failure, **stale cache** served if available (`fromCache`, `isStale` flags).

Frontend: `WeatherScreen`, `CitySearchScreen`, `HomeWeatherCard` CONSUME `WeatherProvider` via `Consumer/context.watch`.

Caching:

- **Hive** `weather_cache` box: keys `snapshot_{lat_lon}` — full `weatherSnapshot` JSON.
- **SharedPreferences:** selected location, saved locations list (max 5), last sync millis.

API endpoints (representative):

Provider	Example endpoint pattern
Open-Meteo forecast	GET <code>{WEATHER_API_BASE_URL}/forecast?latitude=&longitude=&curr</code>
Open-Meteo geocode	GET <code>https://geocoding-api.open-meteo.com/v1/search?name=</code>
Open-Meteo reverse	GET <code>.../reverse?latitude=&longitude=</code>
OpenWeather current	GET <code>/data/2.5/weather?lat=&lon=&appid=</code>
OpenWeather forecast	GET <code>/data/2.5/forecast?... </code>
OpenWeather One Call	GET <code>/data/3.0/onecall?... </code>
OpenWeather geo	GET <code>/geo/1.0/direct</code> and <code>/reverse</code>

Request/response: Standard HTTP GET + JSON. Parsed into domain entities (`CurrentWeather`, `DailyForecast`, etc.).

Farmer advisory: `farmer_advisor.dart` + `WeatherConstants` thresholds (rain %, wind km/h, heat/frost/UV) applied to current snapshot for banners/text.

Auto-refresh: `WeatherProvider.startAutoRefresh()` USES `WeatherConstants.autoRefreshInterval` (**15 minutes**).

6.3 Crop Calendar

Purpose: Display **season timeline** (sowing → irrigation → fertilizer → harvest, etc.) for crops **Wheat, Rice, Cotton** across areas **Multan / Lahore** (Punjab-oriented month ranges).

How “recommendations” are generated: Not AI. CropCalendarCatalog holds static CropCalendarPlan objects with MonthRange windows. CropCalendarRepository.loadPlan() picks exact (crop, area) or falls back to another area. CropCalendarProvider computes currentStageIndex and seasonProgress from today’s date vs those month ranges.

Database: None — all in Dart constants.

APIs: None.

6.4 Marketplace

Purpose: Browse agricultural products (demo), filter by category/company, search, favourites.

Data source: mock_products.dart — compile-time list of Product entities.

Provider: MarketplaceProvider — filters mockProducts, toggles favourites in memory only.

Upload / fetch from server: Not implemented.

6.5 Sensor Integration & DSS (Decision Support)

Purpose: Capture **manual** soil moisture (0–100%) and pH (0–14); show trends; combine **rain probability** from weather module; apply **threshold rules** (defaults + optional Firestore overrides); persist readings and show structured DSS cards (irrigation, soil, risk, monitoring).

How sensors “send data” in this app:

- There is **no Bluetooth/Wi-Fi sensor driver** in code. The user types values in a form.
- On submit, SensorRepository.addReading() writes to Firestore collection **sensor_readings** with FieldValue.serverTimestamp().
- watchRecentReadings() listens to the **latest 20** documents ordered by **timestamp**, maps to SensorReading.

Firestore collections:

Collection	Document	Fields used
sensor_readings	auto-id	soilMoisture, phLevel, crop, rainChancePercent, recommendationSummary, recommendationDetails, recommendationPriority, timestamp
sensor_config	rules	Nested maps: default, wheat, Rice, Cotton with threshold doubles/ints (see SensorRepository.fetchRuleSet)

Admin / control: An administrator updates **sensor_config/rules** in Firebase Console (acts as remote config for thresholds). If missing, SensorRuleSet.fallback() supplies agronomy-style defaults.

DSS logic: Implemented in `sensor_screen.dart` (`_buildRuleBasedRecommendation`) — rule IDs like M1–M4 (moisture/rain), P1–P3 (pH). Priority scored in `_calculatePriority` → HIGH / MEDIUM / LOW.

Weather coupling: `_loadCurrentRainChance()` uses `WeatherRepository.fetchCurrentWeather()` so rain chance aligns with the same weather pipeline as the rest of the app.

6.6 Learning Articles / Videos

Purpose: Educational videos per crop category; optional **crop diseases** catalog for browsing.

YouTube: `YouTubeLearningRepository` calls:

```
GET {youtubeApiBaseUrl}/search?part=snippet&type=video&q=Pakistan+farming+{category}&key=...
```

If **no API key**, demo videos are returned and cached in Hive `learning_cache` as JSON (source: `demo` vs `live`).

Catalog: `crop_diseases_catalog.dart` — static disease entries for navigation/detail screens.

6.7 AI Query (“Ask AI”)

Purpose: Chat-style UI with **speech-to-text** input and **text-to-speech** readback.

Behaviour: On send, after **2 seconds** a bot message is appended using localized string `aiPendingResponse` — **no external AI API**.

6.8 Profile

Purpose: View/edit display name, biometric toggle placeholders, sign-out. Uses `AuthSessionController` and secure storage/local auth packages from `pubspec.yaml` as wired in that screen.

6.9 Notifications

Purpose (UI only): Home app bar opens a bottom sheet listing **three hard-coded strings** (irrigation reminder, weather alert, marketplace offer). Badge shows “3” statically.

No FCM, no local notification scheduler in dependencies for messaging.

6.10 Onboarding

Purpose: First-run experience; sets `onboarding_seen` in Hive `app_preferences`.

7. State Management Summary

Provider	Role
<code>AuthSessionController</code>	Firestore session + guest
<code>LocalizationController</code>	EN/UR per user scope in Hive
<code>ThemeController</code>	Light/dark per user scope
<code>WeatherProvider</code>	Weather snapshot, search, saved cities, auto-refresh

CropCalendarProvider	Selected crop/area + resolved plan
MarketplaceProvider	Filters/favourites for mock products
Feature-local ChangeNotifier	AuthFormProvider where used

8. Local & Cloud Data Stores

8.1 Hive boxes

Box	Content
weather_cache	Serialized WeatherSnapshot per location key
learning_cache	YouTube search cache JSON per category
app_preferences	Onboarding flag, locale_{scope}, theme_mode_{scope}, legacy keys

8.2 SharedPreferences (via shared_preferences)

Weather selected location JSON, saved cities JSON list, last sync millis.

8.3 Firebase

- **Authentication:** user records (managed by Firebase).
- **Firestore:** sensor_readings, sensor_config/rules.

MySQL: N/A.

9. API Catalogue (Implemented)

Authentication: Firebase SDK methods (not REST URLs in app code).

Weather: Open-Meteo and/or OpenWeather HTTP GET URLs constructed in weather_api_service.dart.

YouTube: https://www.googleapis.com/youtube/v3/search (configurable base).

Sensor “API”: Firestore SDK (collection().add, .snapshots(), .doc().get()).

10. Sequence Examples

10.1 Login sequence

10.2 Weather refresh sequence

11. File-by-File Inventory ([lib/](#))

Below, **every Dart file** under [lib/](#) is listed with a one-line purpose and primary links.

Root

File	Purpose
main.dart	App entry: Firebase, Hive boxes, providers, PakFasaApp .
firebase_options.dart	Generated Firebase project keys per platform.

[src/app.dart](#)

File	Purpose
app.dart	MaterialApp : themes, locale, AppRoutes.onGenerateRoute .

[src/core/config](#)

File	Purpose
app_config.dart	Resolves API keys and base URLs (dart-define > JSON asset > defaults).

[src/core/error](#)

File	Purpose
error_logger.dart	Crashlytics init, non-fatal logging, user id for reports.

[src/core/localization](#)

File	Purpose
app_localizations.dart	EN/UR string tables and AppLocalizations.delegate .
localization_controller.dart	Hive-backed locale per user scope.

[src/core/routing](#)

File	Purpose
app_routes.dart	Route names and AuthGate wrappers for protected routes.

[src/core/theme](#)

File	Purpose
app_colors.dart	Brand palette constants.

app_theme.dart	Light/dark ThemeData.
theme_controller.dart	Hive-backed theme mode per user scope.

src/core/widgets

File	Purpose
auth_gate.dart	Route guard: login vs child.
common_states.dart	Shared loading/error UI helpers.
language_toggle_button.dart	Language switch UI widget.
offline_badge.dart	Offline indicator widget.
pakfasal_scaffold.dart	Common scaffold with optional navigation helpers.

src/features/ai_query/presentation/screens

File	Purpose
ai_query_screen.dart	Chat UI; stub bot reply; STT/TTS integration.

src/features/auth/presentation/providers

File	Purpose
auth_session_controller.dart	Firebase auth orchestration + guest mode.
auth_form_provider.dart	Form state for auth screens (if used by signup/login).

src/features/auth/presentation/screens

File	Purpose
login_screen.dart	Email login, guest, navigation to signup/forgot.
signup_screen.dart	Registration + display name.
forgot_password_screen.dart	Password reset email.

src/features/crop_calendar/data

File	Purpose
crop_calendar_catalog.dart	Static plans for crops × Punjab areas.
repositories/crop_calendar_repository.dart	Loads plans from catalog with fallback.

src/features/crop_calendar/domain/entities

File	Purpose
crop_calendar_models.dart	CropType, CropArea, CropCalendarPlan, stages, month ranges.

src/features/crop_calendar/presentation/providers

File	Purpose
crop_calendar_provider.dart	Selected crop/area + derived progress indices.

src/features/crop_calendar/presentation/screens

File	Purpose
crop_calendar_screen.dart	UI for calendar timeline and selectors.

src/features/crop_calendar/presentation/widgets

File	Purpose
area_chip_selector.dart	Region chips.
crop_chip_selector.dart	Crop chips.
season_progress_card.dart	Progress visualization.
sowing_plan_card.dart	Sowing info card.
timeline_stage_tile.dart	Stage row UI.

src/features/crop_calendar/presentation/utils

File	Purpose
crop_calendar_visuals.dart	Helpers/colors for calendar visuals.

src/features/home/presentation/screens

File	Purpose
home_dashboard_screen.dart	Main dashboard: weather card, quick tiles, drawer, bottom nav, notification sheet.

src/features/home/presentation/widgets

File	Purpose
dashboard_tile.dart	Grid tile for module shortcuts.
home_weather_card.dart	Compact weather summary widget.

src/features/learning/data

File	Purpose
crop_diseases_catalog.dart	Static disease content catalog.
repositories/youtube_learning_repository.dart	YouTube search + Hive cache + demo fallback.

src/features/learning/domain/entities

File	Purpose
learning_video.dart	Video metadata entity.

src/features/learning/presentation/screens

File	Purpose
learning_dashboard_screen.dart	Learning hub navigation.
learning_screen.dart	Video list UI for a category.
crop_selection_screen.dart	Pick crop for learning path.
crop_disease_detail_screen.dart	Disease detail view.

src/features/marketplace/data/mock

File	Purpose
mock_products.dart	Demo Product list.

src/features/marketplace/domain/entities

File	Purpose
product.dart	Product model.

src/features/marketplace/presentation/providers

File	Purpose
marketplace_provider.dart	Filters + favourites over mock data.

src/features/marketplace/presentation/screens

File	Purpose
marketplace_screen.dart	Product grid/list + search UI.
product_detail_screen.dart	Detail + contact actions.

src/features/marketplace/presentation/widgets

File	Purpose
product_card.dart	Card UI for a product.

src/features/onboarding/presentation/screens

File	Purpose
------	---------

onboarding_screen.dart	First-run slides + saves onboarding flag.
------------------------	---

src/features/profile/presentation/screens

File	Purpose
profile_screen.dart	User profile and settings actions.

src/features/sensor/data/repositories

File	Purpose
sensor_repository.dart	Firestore CRUD/watch + rule document parsing + SensorRuleSet.

src/features/sensor/domain/entities

File	Purpose
sensor_reading.dart	One sensor sample model.

src/features/sensor/presentation/screens

File	Purpose
sensor_screen.dart	Manual entry, charts, DSS output, TTS, Firestore sync.

src/features/splash/presentation/screens

File	Purpose
splash_screen.dart	Branded splash animation + route to onboarding/login/home.

src/features/weather/constants

File	Purpose
weather_constants.dart	Endpoints paths, TTLs, Hive key names, advisory thresholds.

src/features/weather/domain/entities

File	Purpose
weather_models.dart	Locations, snapshots, forecasts, alerts.

src/features/weather/data/repositories

File	Purpose
weather_repository.dart	Location resolution, caching, saved cities, API orchestration.

src/features/weather/data/services

File	Purpose
weather_api_service.dart	HTTP integrations (OWM + Open-Meteo).
location_service.dart	Geolocator wrapper with structured errors.

src/features/weather/presentation/providers

File	Purpose
weather_provider.dart	UI state machine for weather + city search + auto refresh.

src/features/weather/presentation/screens

File	Purpose
weather_screen.dart	Full weather experience.
city_search_screen.dart	Search + pick saved cities.

src/features/weather/presentation/widgets

File	Purpose
crop_alert_banner.dart	Weather alert strip.
daily_forecast_list.dart	Daily rows.
farmer_advisory_section.dart	Advisory blocks.
hourly_forecast_slider.dart	Hourly carousel.
section_card.dart	Styled section container.
sunrise_sunset_card.dart	Sun times card.
temperature_hero_card.dart	Large temperature display.
weather_app_bar.dart	Weather screen app bar.
weather_error_view.dart	Error UI.
weather_highlights_grid.dart	Metrics grid.
weather_skeleton.dart	Loading placeholders.

src/features/weather/presentation/utils

File	Purpose
farmer_advisor.dart	Advisory text from thresholds + weather state.
weather_gradients.dart	Background gradients by condition.
weather_view_mapper.dart	Maps condition codes to UI/iconography.

12. System Architecture Summary

- **Single Flutter client** talking to **Firebase Auth**, **Firestore** (sensors + remote DSS config), and **public HTTP APIs** (weather, YouTube).
- **Heavy client-side logic** for DSS when manual readings submitted.
- **Offline-first weather** via Hive + stale indicators.

13. Data Flow Summary

1. **Weather:** GPS/prefs → Repository → (cache hit OR HTTP) → `WeatherProvider` → widgets.
2. **Learning:** Repository → (Hive OR YouTube OR demo) → screens.
3. **Sensors:** Form → rule engine + weather rain → Firestore write → stream updates charts.
4. **Auth:** UI → Firebase Auth → `AuthSessionController` listeners → gated routes.

14. Developer Documentation — Operations

Run (see README):

```
flutter pub get cp config/app_config.json config/dev.json # then edit keys flutter run
--dart-define-from-file=config/dev.json
```

Secrets: Never commit real keys; use `config/dev.json` (gitignored) or CI `dart-define`.

Firestore indexes: For `sensor_readings` ordered by `timestamp`, create a **composite index** in Firebase Console if queries require it (the code uses `orderBy('timestamp')`).

15. Future Improvements (Concrete)

1. **Backend marketplace:** REST API + Firestore/Cloud Storage for real listings and image uploads; replace `mock_products.dart`.
2. **Real AI:** Integrate Gemini/OpenAI/self-hosted endpoint; replace stub in `AiQueryScreen`.
3. **Push notifications:** Add `firebase_messaging`, store FCM tokens, Cloud Functions for triggers.
4. **Admin app or Role:** Secure admin UI to edit `sensor_config/rules` without Firebase Console.

5. **Hardware sensors:** BLE/MQTT client or ingest from gateway; map to same `SensorReading` pipeline.
6. **Tests:** Unit tests for `weatherRepository` caching and DSS rule scoring.
7. **Optional MySQL:** Only if you introduce a dedicated backend; currently unnecessary for this client-centric design.

16. Diagram — End-to-End User Journey

End of PakFasal Complete Documentation.